



INSTITUTO SUPERIOR DE ENGENHARIA DE LISBOA (ISEL)

DEPARTAMENTO DE ENGENHARIA INFORMÁTICA (DEI)

MMEIM

MESTRADO EM ENGENHARIA INFORMÁTICA E MULTIMÉDIA
VISÃO ARTÍFICIAL E REALIDADE MISTA

Projeto 1

Miguel Alcobia (50746)

Docente

Professor Pedro Mendes Jorge

Abril, 2026

Índice

Índice	i
Lista de Figuras	ii
1 Introdução	1
2 <i>Face Detection</i>	2
2.1 Implementação	2
3 Face Recognition	4
3.1 <i>Dataset</i>	4
3.2 Algoritmo LBP	4
3.3 Implementação	5
3.4 Normalização	6
4 Virtual Objects	8
4.1 Reconhecimento em Tempo Real	8
4.1.1 Estabilização Temporal (<i>FaceTracker</i>)	9
4.1.2 Aplicação de Filtros de Óculos	10
5 Conclusão	12
Bibliografia	13

Lista de Figuras

2.1	Resultado da detecção da face com o Mediapipe	3
3.1	Antes e depois da normalização	7
4.1	Demonstração da aplicação	11

Capítulo 1

Introdução

Este trabalho tem como objetivo o desenvolvimento de uma aplicação para detecção e reconhecimento facial, tendo também objetos virtuais alinhados com as faces detetadas.

A aplicação foi desenvolvida com a ajuda da biblioteca OpenCV e Mediapipe, que facilitou a detecção de pontos de referência facial e a integração de elementos virtuais.

O desenvolvimento foi dividido em três fases: Detecção da face, reconhecimento da face e adição dos objetos virtuais.

O projeto foi organizado da seguinte forma dentro da pasta raiz `prj1`:

- `att_faces` - pasta onde estão as pastas com as caras do *dataset*.
- `code` - pasta com o código do trabalho em si, o modelo de reconhecimento treinado e o modelo usado pelo Mediapipe.
 - `spikes` - *scripts* usados para explorar o algoritmo LBP e a detecção de rostos com o Mediapipe.

Capítulo 2

Face Detection

Escolheu-se o *mediapipe*, porque, após analisar todos os métodos sugeridos, foi o que chamou mais atenção pelas possibilidades que esta biblioteca oferecia.

Embora tenham sido analisados todos os métodos sugeridos, acabou-se por ficar entre o **Haar Cascade Classifier**, o **dlib** e o **mediapipe**.

O Mediapipe, assim como outras bibliotecas sugeridas, recorre a *machine learning* para detetar rostos humanos, tanto numa imagem estática como num vídeo ao vivo.

O facto do Mediapipe já ter esse treino com *machine learning* e permitir a visualização de *meshes* no rosto, de forma relativamente simples em comparação com as alternativas, contribuiu para que se optasse por esta biblioteca.

2.1 Implementação

Para a deteção do rosto, o código implementado foi baseado nos exemplos apresentados pela Google em [Google, 2026], [Google, 2025] e [GoogleAI, 2026].

Mesmo com o apoio das referências, surgiram algumas dificuldades, porque o módulo **solutions** mudou recentemente de sítio, dentro da hierarquia da biblioteca (passou para o módulo **tasks**), e a maioria dos exemplos mais completos ainda não contemplam essa alteração.

O ficheiro para o *spike* da deteção da face está em **face_det.py** dentro da pasta **spikes**.

O resultado obtido após a exploração do processo para detetar faces com

o Mediapipe está apresentado na imagem abaixo:

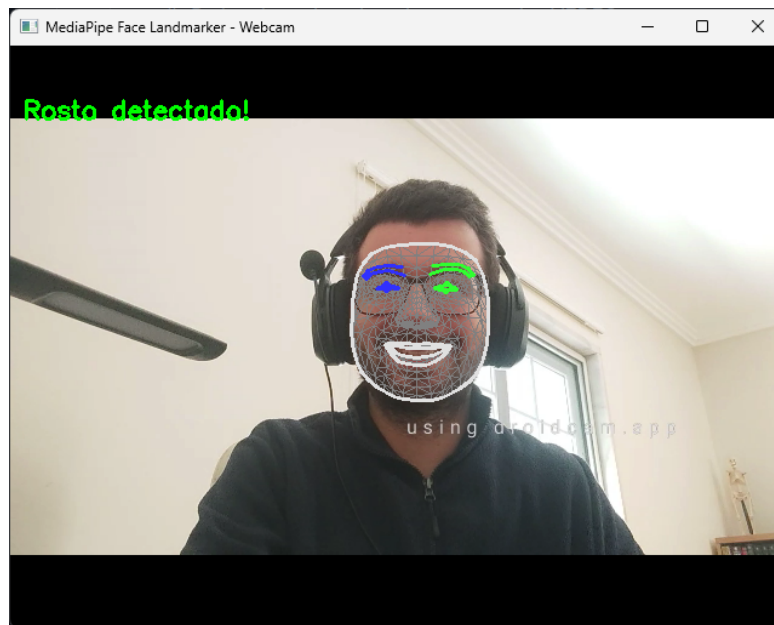


Figura 2.1: Resultado da detecção da face com o Mediapipe

Dado que o Mediapipe foi a abordagem adotada para esta etapa do projeto, a detecção de faces ficou resolvida. De todas as fases, foi a mais simples, dado que o Mediapipe já facilitava bastante o processo com o que oferece.

Capítulo 3

Face Recognition

3.1 *Dataset*

Para conseguir progredir com o trabalho, usou-se o **AT&T Database of Faces** [Damkhang, 2019] até o professor disponibilizar o *dataset* para o projeto.

Assim que foi disponibilizado, acrescentou-se a cara do professor ao conjunto e eliminaram-se a maioria dos rostos da AT&T por não serem necessários. Permaneceram só alguns para efeitos de testes.

3.2 Algoritmo LBP

Após pesquisar e comparar os métodos sugeridos no enunciado, optou-se pelo **Local Binary Patterns (LBP)**.

O LBP é simples de implementar e, apesar da simplicidade, as referências sobre ele referiam a robustez perante a variação de luz nas fotos, o que despertou algum interesse.

O Eigenfaces também despertou alguma curiosidade, mas, pela pesquisa realizada, o algoritmo apresentava a limitação de precisar que todas as fotos fossem tiradas sob as mesmas condições e que fossem do mesmo tamanho [Eigenface, 2026].

Visto que o professor disponibilizaria um *dataset*, o aluno criaria um para si e o trabalho seria validado num terceiro ambiente, não parecia que os requisitos para o bom funcionamento do algoritmo fossem preenchidos. Contudo, mais tarde descobriu-se que esta preocupação não se confirmaria, devido à

normalização.

3.3 Implementação

A parte do algoritmo LBP que foi implementada e a pesquisa para entender o seu funcionamento tiveram por base as referências [PyImageSearch, 2021], [xRay Pixy, 2020] e [CSCoach, 2024].

O LBP começa por passar a imagem para *grayscale*, pois assim diminuimos a informação presente na imagem, mas mantêm-se os padrões que o algoritmo precisa para trabalhar.

Depois, o algoritmo extrai o padrão binário local em cada zona da imagem (cada zona é uma grelha de x por x pixels) onde se começa por calcular a diferença do valor dos pixels vizinhos com o pixel central da zona. Todos os números negativos passam a 0 e todos os números positivos passam a 1 (inclusive o 0).

Por fim, colocamos os números alinhados (começando no canto superior esquerdo e continuando no sentido horário) para formar um número binário. Ao passar o número para decimal, obtemos o identificador daquele padrão local.

Com estes valores, podemos ver que os histogramas que dizem respeito à mesma pessoa apresentam alguma similaridade, e o algoritmo recorre ao cálculo da distância entre histogramas para saber se duas imagens apresentam, ou não, a mesma pessoa. Como apontado no vídeo [CSCoach, 2024] existem várias fórmulas que permitem calcular distâncias entre histogramas e, quanto menor for a distância entre eles, maior é a probabilidade dos histogramas representarem uma foto da mesma pessoa.

Foi implementada uma parte do algoritmo, porque primeiramente pensou-se que o algoritmo tinha de ser feito de raiz, mas depois reparou-se que o enunciado já sugeria uma função do OpenCV para esta finalidade. Porém, a primeira parte do LBP (codificação da imagem e dos valores LBP dos vários blocos que a compõem) foi implementada manualmente e pode ser encontrada na pasta de `spikes`.

A dificuldade seguinte foi encontrar os parâmetros corretos, sobretudo o *threshold*. Experimentaram-se valores que se aproximassem do valor de confiança que fica sob a *bounding box* até encontrar um que fosse suficientemente

restrito, mas deixasse uma margem de confiança.

A biblioteca do OpenCV usa a distância *Chi-Square* como é indicado em [OpenCV, 2018], o que coincide com o exemplo dado no vídeo indicado acima do CS Coach.

3.4 Normalização

As fotos também foram um pequeno problema, devido a não serem todas uniformes.

Numa primeira abordagem, a detecção da face só estava a ser feita no teste e com a webcam, mas devido à falta de uniformidade das fotos no *dataset*, chegou-se à conclusão de que seria melhor também fazer a detecção logo no treino.

Desta forma, as imagens apresentariam apenas a região da face e, consequentemente, a menor variação possível entre as fotos da mesma pessoa.

Após estes problemas, chegou-se à conclusão de que os receios dos problemas que foram levantados perante o Eigenfaces, provavelmente poderiam ser ultrapassados da mesma forma.

A normalização seguiu a sugestão dada pelo enunciado e implementou-se a MPEG-7. Contudo, experimentou-se fazer uma alteração ao fazer uma imagem 200x200 em vez de 46x56.

Com este último tamanho, a confiança andava por volta de 130 a 160 e com 200x200 ficava entre 40 a 70. Porém, mesmo com um valor de distância maior, a resolução de 46x56 apresentava resultados mais estáveis durante o reconhecimento na aplicação.

Desenvolveu-se a função `normalizar_face_mpeg7` que começa por converter a imagem em tons de cinza. Em seguida, são extraídas as *landmarks* obtidas pelo Mediapipe, cujos pontos foram obtidos pela consulta de [Irfan, 2021].

Com as *landmarks* calcula-se a posição vertical média dos olhos, ao calcular a média do ponto médio de cada olho. No passo seguinte, a imagem é redimensionada para a resolução de 46x56. A nova posição dos olhos na imagem redimensionada é calculada ao multiplicar a altura da imagem (56) por 0,4, o que dá para confirmar, visto que $(56 * 24)/100 \approx 40$.

Este cálculo foi feito porque, desta forma, evitava-se trocar sempre o valor da posição dos olhos.

As bordas das imagens são preenchidas, para evitar bordas pretas, com os pixels que delimitam a imagem.

Na imagem abaixo é demonstrada a imagem antes e depois da normalização.

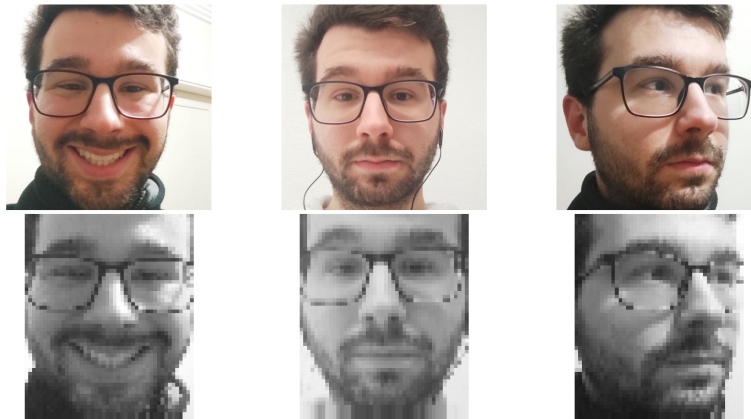


Figura 3.1: Antes e depois da normalização

Capítulo 4

Virtual Objects

4.1 Reconhecimento em Tempo Real

Após o treino do modelo, desenvolveu-se um *script* para fazer tudo o que foi implementado nas fases anteriores, mas agora através da *webcam* e aplicar objetos virtuais (óculos, neste caso) após o reconhecimento ser efetuado.

No começo, três componentes importantes para o resto do processo:

- **Modelo LBPH treinado** — Um ficheiro YAML que é guardado no fim do treino com o classificador treinado.
- **Modelo de deteção facial** — *Face Landmarker* do MediaPipe, que deteta as faces e extrai os 478 pontos (*landmarks*).
- **Imagens dos óculos** — Imagens PNG que vão ser usadas como filtro em formato. São carregadas no início para um array que serve de (*cache*), evitando atrasos durante a execução em tempo real na *webcam* quando eles mudam entre si.

Para cada *frame* capturado pela webcam, o MediaPipe processa a imagem e deteta até 3 pessoas (valor definido em `MAX_PESSOAS` simultaneamente. Para cada face detetada, são devolvidos os 478 *landmarks* com coordenadas normalizadas (valores entre 0 e 1).

Depois, a função `extrair_rosto_normalizado()` recorta a região da face a partir dos *landmarks*, adiciona um pequeno *padding* de 10 pixels e aplica a normalização MPEG-7 com a ajuda do método `normalizar_face_mpeg7()` que segue a implementação referida no capítulo 3.

No fim deste processo, já temos uma imagem que pode ser classificada pelo classificador do LBPH. Esta classificação devolve:

- `label_id` — Identificador da pessoa
- `confianca` — Valor da distância entre os histogramas, como foi explicado no capítulo 3.

Se a confiança for igual ou superior ao threshold definido (`THRESHOLD_CONFIANCA`), que foi posto a 165, a face é considerada desconhecida e o `label_id` é mudado para 8. Esta troca dá-se, pois a label 8 era do rosto do *dataset* do AT&T e ficou apontada para ser a classe que representa um desconhecido.

4.1.1 Estabilização Temporal (*FaceTracker*)

Para evitar classificações instáveis, devido a movimentos, expressões faciais ou a condições de iluminação, implementou-se uma classe `FaceTracker` que acumula as classificações durante um período de estabilização (5 segundos). Durante este período:

- Todas as classificações são guardadas numa lista;
- A interface exhibe uma barra de progresso;
- A classificação mostrada é temporária (pode mudar a cada *frame*).

Após os 5 segundos, o *tracker* calcula:

- A label mais frequente entre todas as classificações do período;
- A confiança média dessa label;
- Se a confiança média for maior ou igual ao threshold, a face é marcada como "Desconhecido".

A partir deste momento, o reconhecimento acaba e a *bounding box* fica verde (conhecido) ou laranja (desconhecido), sem novas alterações enquanto a face permanecer na cena.

4.1.2 Aplicação de Filtros de Óculos

Para cada face reconhecida, sobrepõe-se um par de óculos específico do dicionário OCULOS_PESSOA).

Utilizando as *landmarks* faciais, calcula-se:

- **Largura dos óculos** — baseada na distância entre os cantos externos dos olhos
- **Altura dos óculos** — proporcional à largura, seguindo o fator de 40% que se usou para altura dos óculos
- **Posição horizontal** — centralizada sobre os olhos
- **Posição vertical** — alinhada com a altura média dos olhos, com um *shift* específico para quando a abertura dos olhos é inferior a 15 pixels, o que evita que os óculos subam ao pestanejar.

Em seguida, recorre-se à função `calcular_rotacao_olhos()` que determina o ângulo de rotação dos óculos com base em dois fatores:

- **Inclinação da cabeça** — calculada através do ângulo entre os centros dos olhos
- **Inclinação vertical** — Calculada pela posição normalizada dos olhos relativamente ao topo da cabeça e ao queixo. Se os olhos estão acima de 35% (cabeça inclinada para cima) ou abaixo de 45% (cabeça inclinada para baixo), aplica-se uma correção de ± 0.2 pixels que é multiplicada por 15 graus para dar uma suavização. Estas contas têm por base que, como já foi referido, os olhos com o MPEG-7 devem estar a 0.4

O ângulo final é dado por:

$$\text{angulo}_{\text{oculos}} = -\text{angulo}_{\text{cabeça}} + (\text{fator}_{\text{vertical}} \times 15)$$

Para evitar tremores na rotação dos óculos, aplica-se um filtro exponencial com um fator de suavização 0.3 [Busarello, 2020]:

$$\text{angulo}_{\text{suavizado}} = (0.3 \times \text{angulo}_{\text{anterior}}) + (0.7 \times \text{angulo}_{\text{atual}})$$

Sobreposição com Transparência

A sobreposição dos óculos é feita na função `sobrepor_imagem_com_transparencia()` que realiza os seguintes passos:

- Redimensionamento da imagem PNG para as dimensões calculadas;
- Aplicação da rotação com a função `rodar_img()` [GeeksforGeeks, 2024a], que recalcula automaticamente as dimensões do *canvas* para evitar cortes;
- Extração dos canais BGR e *alpha* (transparência);
- *Alpha blending* na região de interesse do *frame* original (zona onde os óculos são colocados) [Wikipedia, 2001]:

$$\text{roi} = (1 - \alpha) \times \text{roi}_{\text{fundo}} + \alpha \times \text{bgr}_{\text{óculos}}$$

- Garante que os óculos não ultrapassem os limites do *frame*.

Abaixo está o resultado final da aplicação (`cam_rec_glasses_rot_temp.py`), já depois de reconhecer um rosto e com os respetivos óculos.

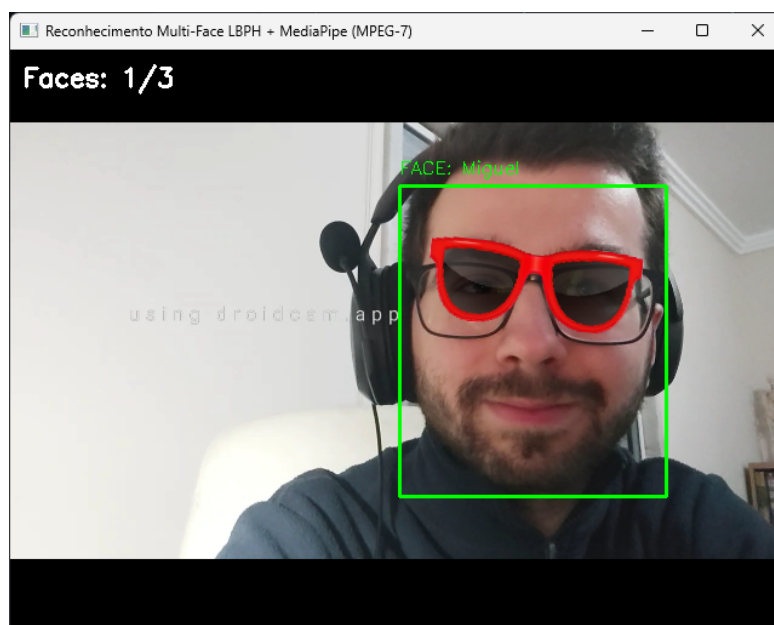


Figura 4.1: Demonstração da aplicação

Capítulo 5

Conclusão

O trabalho serviu para conhecer mais aprofundadamente como funcionam os algoritmos e sistemas para reconhecimento facial, tal como são implementados filtros que estamos habituados a ver em redes sociais (embora este projeto seja uma versão simplificada disso).

A maior dificuldade encontrada no trabalho foi fazer os ajustes necessários, tanto às fotos como aos parâmetros, para que a classificação fosse o mais precisa possível.

Apesar da classificação, com alguma frequência dar "Desconhecido" ou confundir pessoas parecidas, acredito que a estabilização de 5 segundos ajudou a minimizar o problema e o facto de termos sempre os óculos a mudar.

Foi interessante realizar a pesquisa que apoiou o trabalho e descobrir novos algoritmos e técnicas que desconhecia.

Bibliografia

- [Busarello, 2020] Busarello, T. D. C. (2020). Filtros de média - parte 4. <https://www.youtube.com/watch?v=R-EdyhA5jYw&t=10s>. Acedido: 2026.
- [CSCoach, 2024] CSCoach (2024). How facial recognition works — local binary patterns explained. <https://www.youtube.com/watch?v=6zX7YhNRH44>. Acedido: 2026.
- [Damklian, 2019] Damklian, K. (2019). At&t database of faces. <https://www.kaggle.com/datasets/kasikrit/att-database-of-faces?resource=download>. Acedido: 2026.
- [Eigenface, 2026] Eigenface, W. (2026). Eigenface. <https://en.wikipedia.org/w/index.php?title=Eigenface&oldid=1335766723>. Acedido: 2026.
- [GeeksforGeeks, 2020] GeeksforGeeks (2020). Python opencv - affine transformation. <https://www.geeksforgeeks.org/python/python-opencv-affine-transformation/>. Acedido: 2026.
- [GeeksforGeeks, 2024a] GeeksforGeeks (2024a). Rotation matrix. <https://www.geeksforgeeks.org/maths/rotation-matrix/>. Acedido: 2026.
- [GeeksforGeeks, 2024b] GeeksforGeeks (2024b). Transformation matrix. <https://www.geeksforgeeks.org/maths/transformation-matrix/>. Acedido: 2026.
- [Google, 2025] Google (2025). Guia de detecção de pontos de referência do rosto. https://ai.google.dev/edge/mediapipe/solutions/vision/face_landmarker?hl=pt-br. Acedido: 2026.

- [Google, 2026] Google (2026). Guia de detecção de pontos de referência facial para python. https://ai.google.dev/edge/mediapipe/solutions/vision/face_landmarker/python?hl=pt-br. Acedido: 2026.
- [GoogleAI, 2026] GoogleAI (2026). Mediapipe samples. <https://github.com/google-ai-edge/mediapipe-samples/tree/main>. Acedido: 2026.
- [Irfan, 2021] Irfan, M. (2021). simplified_mediapipe_face_landmarks. https://github.com/k-m-irfan/simplified_mediapipe_face_landmarks. Acedido: 2026.
- [OpenCV, 2018] OpenCV (2018). Face recognition based on lbp. https://github.com/opencv/opencv_contrib/blob/master/modules/face/src/lbph_faces.cpp. Acedido: 2026.
- [PyImageSearch, 2021] PyImageSearch (2021). Face recognition with local binary patterns (lbps) and opencv. <https://pyimagesearch.com/2021/05/03/face-recognition-with-local-binary-patterns-lbps-and-opencv/>. Acedido: 2026.
- [Wikipedia, 2001] Wikipedia (2001). Alpha compositing. https://en.wikipedia.org/wiki/Alpha_compositing#History. Acedido: 2026.
- [xRay Pixy, 2020] xRay Pixy, R. (2020). How is the lbp —local binary pattern— values calculated? <https://www.youtube.com/watch?v=h-z9-bMtd7w>. Acedido: 2026.